RAG智能问答系统实战

理解了文本向量化之后，我们可以开始构建一个RAG智能问答系统了。通过RAG可以给大模型灌输一些客户自己的资料，然后让大模型能够尝试理解客户的自然提问后，结合灌输的资料给出更合理，更精确的答案。接下来，我们将会整理一套美团的常见问题说明文档，整理给大模型，让大模型能够更好的理解美团的重要业务流程。

> 说明：通常RAG更适合用于处理客户自己的，不太适合对互联网公开的资料。这个例子中，我们用国家规定做为案例，仅仅只是用来介绍RAG的用法。

1、RAG基础流程

如果我们直接询问AI大模型，它会直接给出答案，但是答案可能不够准确，或者不够详细。例如：

```
from langchain_openai import ChatOpenAI
from config.load_key import load_key

llm = ChatOpenAI(
    model="qwen-plus",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key=load_key("BAILIAN_API_KEY"),
)

llm.invoke("如何退款？")
```

虽然大模型也依然能够给出一个答案，但是，很显然，他并没有专门“学习”美团的业务知识，自然就无法针对美团的常见问题说明，给出比较理想的答案。

那么，如何让大模型能够“学习”美团的业务知识呢？典型的方法有两种：

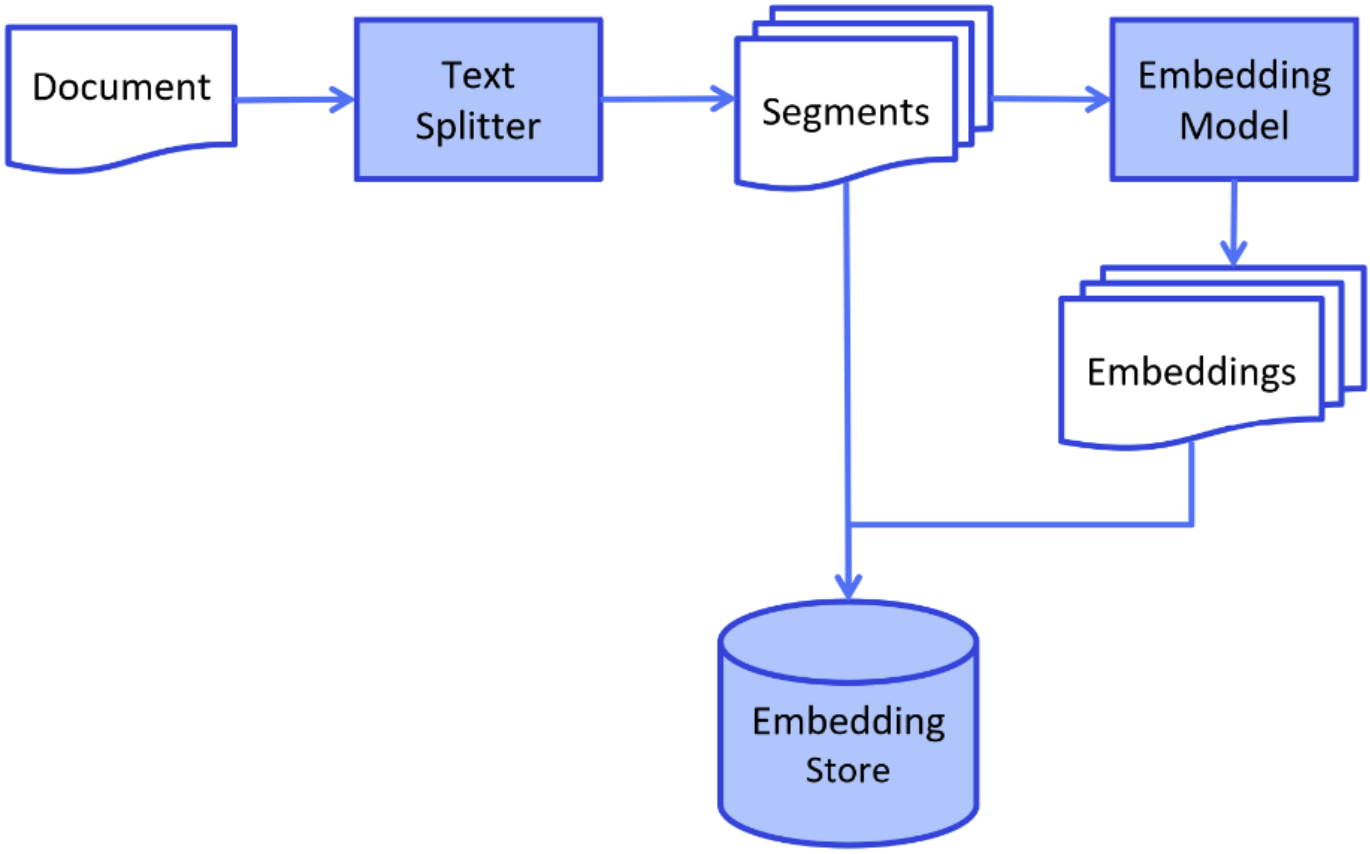
1. 一种方法是微调。就是让大模型回炉进行一次学习，你可以理解为让一个已经毕业的博士，回炉重新学习美团的业务知识。通常像通义千问、deepseek这样的大模型，经过复杂，庞大的学习过程，已经具备了非常好的理解能力和知识储备，这些称为预训练模型。pretraining。 而微调就是在这些预训练模型的基础上，再投入少量的资源，让大模型继续学习。微调的过程，就是让大模型学习到我们自己收集的数据，让大模型“学习”到我们自己收集的数据，从而达到更好的理解能力。这种方式针对性比较强，对问题的立即恩也能更精确。但是，随之而来的问题是成本还是比较高，而且实现的难度比较大。如果没有足够的功底，甚至可能让大模型产生倒退。
2. 另一种方法就是RAG。Retrival-Augmented Generation 检索增强生成。这种方法就是在询问大模型问题时，先检索出跟问题可能相关的参考信息，然后再把问题跟参考信息一起输入给大模型，让大模型结合参考信息，给出更好的答案。你可以理解为我们去向一个专家询问相关业务问题时，带上业务手册。这样专家就不用提前学习额外的知识，只需要结合业务手册，就能快速给出针对性的帮助。这种方式成本比较低，也更适合处理那些涉及到大量外部数据的特定问题。因此，RAG也是目前企业用得做多的一种方法。

> 通常，RAG和模型微调也是可以混合使用的。RAG是给大模型锻炼四肢，而模型微调是给大模型补脑子，两者相辅相成。

RAG的基础工作流程通常分为两个阶段：Indexing索引阶段 和 Retrival检索阶段：

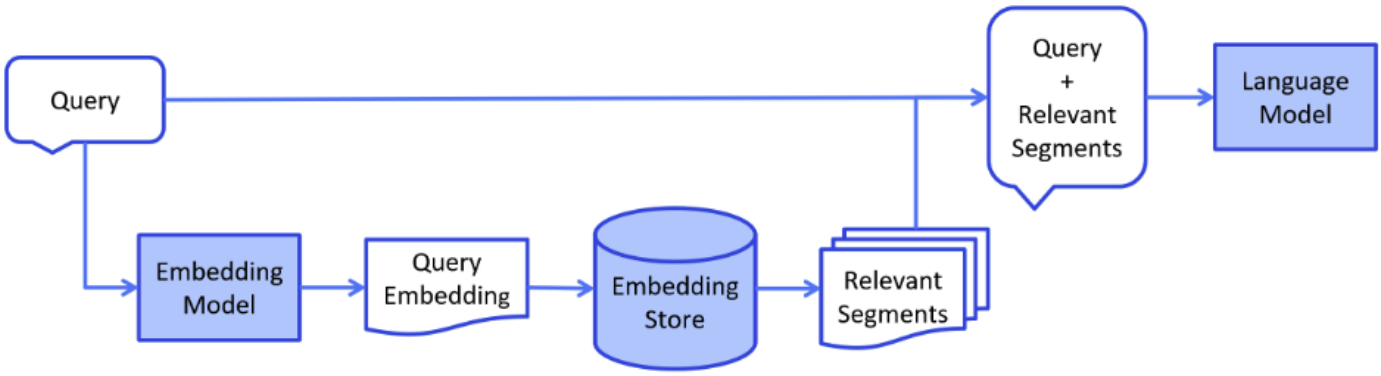
- indexing索引阶段：

这一阶段主要是要对相关文档进行处理，形成知识库，便于后续检索。通常需要将各种形式的文档转化成为Document，然后将Document拆分成小段的Segments，然后将这些Segments进行Embedding向量化处理，并将结果保存到向量数据库当中，这样，后续的检索工作就可以直接使用向量数据库进行检索了。



- Retrival检索阶段

这一阶段主要是当用户提出一个问题时，可以到向量数据库中检索出跟用户的问题比较关联的Segment。把这些segment和用户的问题一起整理成完整的prompt，再发送给大模型。然后由大模型对信息金正整合，再给用户返回正确答案。



> 例如可以定制这样的一个prompt模板：
>
> prompt_template = "*"
> 你是一个问答机器人。
> 你的任务是根据下述给定的已知信息回答用户问题。
> 已知信息：
> ｛context｝ # ｛context｝就是检索出来的文档
> 用户问：
> ｛question｝ # ｛question｝ 就是用户的问题

> 如果已知信息不包含用户问题的答案，或者已知信息不足以回答用户的问题，请直接回复“我无法回答您的问题”。
> 请不要输出已知信息中不包含的信息或答案。
> 请用中文回答用户问题。

这两个阶段都需要做很多精细化的工作，才能保证RAG最终能够得到比较理想的结果，甚至还可能需要做多次的调整。而LangChain框架针对这两个阶段，都提供了很多工具。接下来，我们一步步实战把。

2、Indexing索引阶段

1-加载并解析文档

langchain中提供了非常多Document Loader工具，可以从PDF、HTML、MarkDown、JSON、CSV等各种格式的文档中加载数据。甚至还实现了非常多的扩展工具，可以从网页上加载数据。使用这些工具，可以很方便的加载文档。例如：

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("./resource/meituan-questions.txt")
documents = loader.load()
documents
```

在LangChain中，TextLoader是BaseLoader的一个实现类。除了TextLoader外，还有非常多的实现类。例如，如果你的知识库文件比较多，你可以尝试使用DirectoryLoader，它会自动递归的加载文件夹中的所有文件。

```
from langchain_community.document_loaders import DirectoryLoader

directLoader = DirectoryLoader("./resource/", glob="**/*.txt",loader_cls=TextLoader,
show_progress=True)
directLoader.load()
```

2-切分文档

接下来，需要将Documents中的文件切分成一个个比较独立的Segments。一个Segments表示一个比较独立的知识片段。例如，对于我们这个示例，可以把一个问答就当成一个segment。实现时，就是按照"\n\n"两个换行符进行切分。

```
from langchain_text_splitters import CharacterTextSplitter

# 切分文档
text_splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=0
,separator="\n\n",keep_separator=True)

segments = text_splitter.split_documents(documents)
print(len(segments))
for segment in segments:
    print(segment.page_content)
    print("--------")
```

测试时，你会发现，CharacterTextSplitter组件切分出来的结果可能和预期结果相比没有那么精确。文档中有31个问答条目，而切分的结果只有6个。这是因为在做RAG文档切分时，通常我们并不需要像传统数据库那样严格的切分文档。最终基于大模型强大的理解能力，即便不是很合理的拆分，也能得到比较好的效果。
当然，如果想要按照每个单独的问答进行严格的切分，也不是没有办法。我们也可以自行切分内容.

```
import re

# 自行切分文档
texts = re.split(r"\n\n", documents[0].page_content)
segments = text_splitter.split_text(documents[0].page_content)
#将文档片段转换成为documents
segment_documents = text_splitter.create_documents(texts)
# 这样得到的就是31个精确的问答
print(len(segment_documents))
for segment in segment_documents:
    print(segment.page_content)
    print("--------")
```

3-文本向量化

切分出我们需要的知识条目后，就可以对文本进行向量化，并将这些向量化的结果保存到向量数据库当中。这里还是以之前介绍过的Redis作为示例

```
import os
from langchain_community.embeddings import DashScopeEmbeddings
from config.load_key import load_key
#构建向量化模型
if not os.environ.get("DASHSCOPE_API_KEY"):
  os.environ["DASHSCOPE_API_KEY"] = load_key("BAILIAN_API_KEY")
embedding_model = DashScopeEmbeddings(model="text-embedding-v1")

#使用Redis构建向量数据库
redis_url = "redis://localhost:6379"

from langchain_redis import RedisConfig,RedisVectorStore
config = RedisConfig(
```

```
    index_name="meituan-index",
    redis_url=redis_url
)
vector_store = RedisVectorStore(embedding_model,config=config)
#文档保存到向量数据库中
vector_store.add_documents(segment_documents)
```

到这里就完成了RAG的第一个阶段，Indexing索引阶段。也就是说知识库内容处理完成。接下来就可以进入第二个阶段Retrival，对客户问题做检索增强了。
最后，把这些代码整合到一起，总结一下RAG建立本地消息索引的过程。

```
from langchain_community.document_loaders import TextLoader
#1、加载原始文档
loader = TextLoader("./resource/meituan-questions.txt")
documents = loader.load()
#2、切分文档
import re
from langchain_text_splitters import CharacterTextSplitter
text_splitter = CharacterTextSplitter(chunk_size=500, chunk_overlap=0
,separator="\n\n",keep_separator=True)

texts = re.split(r"\n\n", documents[0].page_content)
segments = text_splitter.split_text(documents[0].page_content)

segment_documents = text_splitter.create_documents(texts)
#3、将文档向量化，保存到Redis中
from langchain_community.embeddings import DashScopeEmbeddings
from config.load_key import load_key

if not os.environ.get("DASHSCOPE_API_KEY"):
  os.environ["DASHSCOPE_API_KEY"] = load_key("BAILIAN_API_KEY")
embedding_model = DashScopeEmbeddings(model="text-embedding-v1")

redis_url = "redis://localhost:6379"
from langchain_redis import RedisConfig,RedisVectorStore
config = RedisConfig(
    index_name="meituan-index",
    redis_url=redis_url
)

vector_store = RedisVectorStore(embedding_model,config=config)
vector_store.add_documents(segment_documents)
```

3、Retrival检索增强阶段

在这个阶段，主要是要围绕客户提出的问题做一些补充和优化。在接收到用户的一个问题后，我们需要先到向量数据库中去检索一下跟用户提出的问题相关的知识条目。这样未来就可以把用户的问题和本地知识库中相关的知识条目一起发给大模型，让大模型综合考虑之后，给出一个理想的答案。

4-检索相关信息

比如，用户询问“在线支付取消订单后钱怎么返还”这样的问题，我们就需要先到Redis中检索一下跟这个问题相关的知识条目有哪些。

```
query="在线支付取消订单后钱怎么返还"

from langchain_community.embeddings import DashScopeEmbeddings
from config.load_key import load_key

if not os.environ.get("DASHSCOPE_API_KEY"):
  os.environ["DASHSCOPE_API_KEY"] = load_key("BAILIAN_API_KEY")
embedding_model = DashScopeEmbeddings(model="text-embedding-v1")

from langchain_redis import RedisConfig,RedisVectorStore
redis_url = "redis://localhost:6379"
config = RedisConfig(
    index_name="meituan-index",
    redis_url=redis_url
)

vector_store = RedisVectorStore(embedding_model,config=config)
retriever = vector_store.as_retriever()
relative_segments = retriever.invoke(query,k=5)
relative_segments
```

5-构建prompt提示词

查询出跟用户问题相关的“知识”后，就需要将用户的问题和相关的“知识”整合到一起，才能发给大模型。下面是一个比较简单的模板示例

```
from langchain_core.prompts import ChatPromptTemplate
prompt_template= ChatPromptTemplate.from_messages([
    ("user", """你是一个答疑机器人，你的任务是根据下述给定的已知信息回答用户的问题。
    已知信息：{context}
    用户问题：{question}
    如果已知信息不包含用户问题的答案，或者已知信息不足以回答用户的问题，请直接回复“我无法回答您的问题”。
    请不要输出已知信息中不包含的信息或答案。
    请用中文回答用户问题。""")
])

text = []
for segment in relative_segments:
    text.append(segment.page_content)

prompt = prompt_template.invoke({"context":text,"question":query})
prompt
```

```
from langchain_core.prompts import ChatPromptTemplate
prompt_template = ChatPromptTemplate.from_messages([
    ("system","Translate the following from English into {language}"),
    ("user","{text}")
])
prompt = prompt_template.invoke({"language":"Chinese","text":"Hello, how are you?"})
prompt
```

6-调用大模型

接下来，就可以把这个包含了背景知识和用户问题的prompt，发送给大模型了。

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(
    model="deepseek-v3",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key=load_key("BAILIAN_API_KEY"),
)
response = llm.invoke(prompt)
print(response.content)
```

到这里，整个RAG的流程就结束了。我们也可以把Retrival的流程用LCEL语法链起来，这样，你只需要定义检索过程中的各种组件，实现流程会简洁很多。

```
from langchain_core.output_parsers import StrOutputParser

query="在线支付取消订单后钱怎么返还"

from langchain_community.embeddings import DashScopeEmbeddings
from config.load_key import load_key

if not os.environ.get("DASHSCOPE_API_KEY"):
  os.environ["DASHSCOPE_API_KEY"] = load_key("BAILIAN_API_KEY")
embedding_model = DashScopeEmbeddings(model="text-embedding-v1")

from langchain_redis import RedisConfig,RedisVectorStore
redis_url = "redis://localhost:6379"
config = RedisConfig(
    index_name="meituan-index",
    redis_url=redis_url
)
# 定义向量数据库
vector_store = RedisVectorStore(embedding_model,config=config)
retriever = vector_store.as_retriever()

# 定义大模型
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(
    model="deepseek-v3",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
```

```
    openai_api_key=load_key("BAILIAN_API_KEY"),
)
# 定义提示词模板
from langchain_core.prompts import ChatPromptTemplate
prompt_template= ChatPromptTemplate.from_messages([
    ("user", """你是一个答疑机器人，你的任务是根据下述给定的已知信息回答用户的问题。
    已知信息：{context}
    用户问题：{question}
    如果已知信息不包含用户问题的答案，或者已知信息不足以回答用户的问题，请直接回复“我无法回答您的问题”。
    请不要输出已知信息中不包含的信息或答案。
    请用中文回答用户问题。""")
])
# 收集document的内容
def collect_documents(segments):
    text = []
    for segment in segments:
        text.append(segment.page_content)
    return text

# retriever.invoke(query,k=5)
from operator import itemgetter
chain = ({
    "context":itemgetter("question") | retriever | collect_documents,
    "question":itemgetter("question")
    }
    |prompt_template
    |llm
    |StrOutputParser()
)

response = chain.invoke({"question":query})
response
```